

硬件计时器待改动需求

v2026.05.14 | 双向通讯一致性核查结论

背景：PC 控制端在 C:\代码\swiming_claude，硬件固件在 C:\硬件游泳计时器修改程序2026年5月文件夹\H753SWIM2026-5-11。我们对二者逐条核对后，发现以下三项需要硬件侧补做。修改后 PC 端**不需要再改任何代码**——PC 的解析/显示逻辑均已就位。

PC 端已自行修复一项关键 Bug：原 PC 发"盲表 2/3 扩展位图"用了 0x49 / 0x4A，被硬件 dispatcher 当成 Set_DateTime / Set_PoolSingleOrDoubleTP 误处理。已注释。详见本文末尾"背景信息"。

1. 必修 0x1A StartingBlock 抢跳符号位

现状

硬件出发台触发上行 0x1A 帧时，D10 (SW_Back1_Pos, 第 11 字节) 始终为 0。PC 端按 D10 ≠ 0 判抢跳并把反应时取负显示，因此 PC 永远收不到正确的抢跳标志，反应时永远显示为正值。

修改

代码位置	APP/swimplay.c 约 3588 行，发送 0x1A 帧的位置
触发条件	检测出运动员抢跳/犯规（出发台离台时刻早于发令枪）
需做的事	给 TXD_Buffer[SW_Back1_Pos] 赋非零值

```
// 在抢跳分支里加一行
TXD_Buffer[SW_Back1_Pos] = 1;    // 0x01 即可，PC 仅判非 0

// 如果是按“运动员出发时刻 - 发令枪时刻”得到的差值符号判断：
if (diff < 0) TXD_Buffer[SW_Back1_Pos] = 1;
else          TXD_Buffer[SW_Back1_Pos] = 0;
```

PC 端配合（已就位）

```
// TimingBridge.cs:340
bool isFalseStart = false;
if (cmd == 0x1A && rawD10 != 0) {
    isFalseStart = true;
    timeInSeconds = -timeInSeconds;
}
```

2. 必修 0x4B BatteryVoltage 周期上报

现状

APP/swimplay.c:1733 仅有一行注释提及，**完全没有实现** ADC 采样和发送代码。PC 端顶部状态栏的电压显示永远为空。

协议规约

方向	硬件 → PC（硬件主动周期上报）
命令字节 D2	0x4B（直接发送，无 +0x10 偏移）
字段	D3 = mV 高字节，D4 = mV 低字节（大端），D5-D10 = 0

硬件电池：7.4V 标称（2 节锂电串联包，单节 3.7V）。满充 8.4V / 标称 7.4V / 低 7.0V / 耗尽 6.0V。

示例：7.85 V = 7850 mV = 0x1EAA
F1 53 4B 1E AA 00 00 00 00 00 00 F4

USER/swimtime.h 已有定义：#define Report_Voltage_RAW 0x4B

修改建议

在 swimplay.c 主循环或定时任务里，按 5-10 秒一次的节拍上报：

```
static u32 last_volt_send_tick = 0;
if (HAL_GetTick() - last_volt_send_tick >= 5000) {
    last_volt_send_tick = HAL_GetTick();
    u16 mv = ReadBatteryMV();           // 用户已有的 ADC 接口
    u8 buf[12];
    buf[0] = SOH;                       // 0xF1
    buf[1] = 'S';                       // 0x53
```

```

    buf[2] = Report_Voltage_RAW;           // 0x4B ← 直接写，不 +0x10
    buf[3] = (mv >> 8) & 0xFF;           // mV 高字节
    buf[4] = mv & 0xFF;                   // mV 低字节
    buf[5] = 0; buf[6] = 0; buf[7] = 0;
    buf[8] = 0; buf[9] = 0; buf[10] = 0;
    buf[11] = EOT;                         // 0xF4
    TCPIP_Send(buf, 12);                   // 或者你已有的统一发送函数
}

```

PC 端配合 (已就位)

```

// TimingBridge.cs case 0x4B → BatteryVoltage
// MainWindow.xaml.cs ProcessTimingDataFromHardware:
mv = (D3 << 8) | D4
V = mv / 1000.0

// 顶栏显示 + 阈值染色 (2026-05-13(2) 按硬件 7.4V 包 / 2节锂电对齐):
// < 7.00 V          红色 (严重低电, 请立即更换/充电; 接近耗尽 6.0V)
// 7.00 V - 7.40 V    琥珀色 (偏低, 注意监视)
// ≥ 7.40 V          绿色 (健康; 标称 7.4V, 满充 8.4V)

```

3. 选修 0x42 SetCommand 新增子码 0x09

现状

PC → 硬件 已通过 0x41 帧 D5 把"盲表代替成绩延迟"下发, 硬件已接收
(swimplay.c:5447 MBdelay_Time = RXD[5]*10)。主路径 OK。

USER/swimtime.h 已有定义: #define SetCmd_Sub_BlindReplaceDelay 0x09

缺口

硬件想"主动通知 PC 该参数变了"时, 需经 0x42 子码路径。当前 case 0x32 的子码 switch 表 没有 0x09 分支。

修改 (仅当硬件有"GUI 改参数→回传 PC"需求时做)

```

// 在 case 0x32 的子码 switch 里加:
case SetCmd_Sub_BlindReplaceDelay: // 0x09
    // D4 = 0.1秒单位, 例如 50 = 5.0s

```

```
MBdelay_Time = RXD_Data_Buffer[4] * 10;
break;
```

PC 端 `ApplyHardwareSettingFrame:2323` 已就位接收。
若硬件没有"GUI 改参数后回传 PC"的需求，本项可不做。

4. 必修 **0x47 Set_LaneOpenClose 道次打开/关闭** 2026-05-14 新增

现状

PC 比赛控制 → 泳道控制 已经把"全部打开 / 全部关闭 / 道次打开 / 道次关闭" 4 个按钮接到了 `0x47` 命令上。硬件 dispatcher 目前没有 `0x37` case (= `0x47-0x10`)，所以收到后被忽略，按下按钮硬件无反应。

协议规约

方向	PC → 硬件
命令字节 D2	<code>0x47</code> (硬件 case = <code>0x37</code> = <code>0x30</code> + 7)
D3	泳道号 · <code>0xFF</code> = 全部泳道 <code>0x00</code> - <code>0x09</code> = 指定单道
D4	<code>1</code> = 打开 (该道纳入比赛，触板/出发台/盲表按正常状态机响应) <code>0</code> = 关闭 (该道完全屏蔽，硬件不接受其任何信号，相当于空泳道)
D5-D10	0

```
全部道打开：F1 53 47 FF 01 00 00 00 00 00 00 F4
全部道关闭：F1 53 47 FF 00 00 00 00 00 00 00 F4
第 4 道打开：F1 53 47 04 01 00 00 00 00 00 00 F4
第 4 道关闭：F1 53 47 04 00 00 00 00 00 00 00 F4
```

与 0x4C 的语义区分

0x47 道次开/关 → 泳道整体启用/禁用 (关闭后该道相当于空泳道)
0x4C 全开/恢复 → 设备状态机覆盖 (接受所有信号 vs 按状态过滤)
两者独立。`0x47` = 关 优先级最高，被关闭的道无视 `0x4C` = 全开 也不上报。

修改建议

```
// USER/swimtime.h 加
#define Set_LaneOpenClose    0x37    // wire 0x47

// swimplay.c case 列表加
case Set_LaneOpenClose: {
    u8 target = RXD_Data_Buffer[3];    // 0xFF=ALL 0..9=单道
    u8 active = RXD_Data_Buffer[4];    // 1=纳入比赛 0=屏蔽
    u8 a, b, li;
    if (target == 0xFF) { a = 0; b = 10; }
    else if (target < 10) { a = target; b = target + 1; }
    else break;
    for (li = a; li < b; li++) {
        LaneEnabled[li] = active;
        // active=1 → 该道触板/出发台/盲表按正常状态机响应
        // active=0 → 屏蔽该道所有信号（触板上报丢弃、起跳不计反应时）
        // 建议同步刷新该道在 LCD 上“道次”按钮颜色
    }
    break;
}
```

PC 端配合（已就位）

```
// TimingBridge.cs:522
public void SendLaneOpenClose(int laneIndex, bool laneOpen) {
    byte d3 = (laneIndex < 0 || laneIndex >= 10) ? (byte)0xFF : (byte)laneIndex;
    byte d4 = (byte)(laneOpen ? 1 : 0);
    SendFullFrame(0x47, d3, d4);
}

// MainWindow.xaml.cs
// OpenAll_Click / CloseAll_Click / SetSingleLaneOpen 4 个入口已串到 0x47
```

5. 背景信息 PC 端已自行修复的内容

Bug 描述

PC 同步设备状态时连续发三帧：

PC 发的 wire 字节	硬件 D2 – 0x10	硬件 case	正确？
---------------	--------------	---------	-----

0x46	0x36	Set_TPSBMB_State	✓ 正确
0x49	0x39	Set_DateTime	✗ 误改 RTC
0x4A	0x3A	Set_PoolSingleOrDoubleTP	✗ 误切单端

实际后果：每次参数设置 OK、触板模式切换、按 Ready 等，硬件都被误切到"单边模式" (D3 ≠ 0 时 PoolSingleOrDoubleTPbit = 1) → 右端信号全部屏蔽 → 触板/出发台/盲表全部失效。甚至 RTC 也可能被误改。

PC 修复方案

已注释 MainWindow.xaml.cs:2591-2602 两条 SendFullFrame(0x49...) / SendFullFrame(0x4A...) 。硬件侧不用做任何事。

给硬件团队的提醒：如果以后硬件想扩展"盲表 2/3 状态位图"功能，请使用 ≥ 0x60 段的命令码 (避开当前 dispatcher 的 0x30+N 段) 。届时通知 PC 端，我们会重新启用对应的发送。

6. 测试建议

抢跳符号位

- 1. PC 上按"准备就绪"按钮，进入 Ready 状态
- 2. 在硬件上模拟"运动员早于发令枪"踩出发台
- 3. PC 端应显示反应时为**负值** (如 -0.020s) ，且日志里出现 ⚠ 抢跳 (硬件确认)

电池电压

- 1. PC 启动并连上硬件
- 2. 等待 5-10 秒
- 3. 顶部状态栏 TimingStatusText 应出现 | 电池: X.XX V ，色彩按阈值染 (<7.00 红 / 7.00-7.40 琥珀 / ≥7.40 绿)
2026-05-13(2) 按硬件 7.4V 电池包对齐

子码 0x09 (如选做)

- 1. 硬件 GUI 改"盲表代替成绩延迟"参数
- 2. 硬件发 0x42 帧: D3=0x09 D4=新值*10

3. PC 端 AddLog 应出现 硬件参数回报：D3=0x09 D4=...

道次打开/关闭 0x47

- 1. PC 比赛控制 → 泳道控制 → 按"全部关闭"
- 2. 硬件应当把全部 10 道置为"屏蔽"状态 (LCD 道次按钮变灰)
- 3. 摸任意一道触板/盲表 → PC 端不应收到 0x16/0x17/0x18/0x19 帧
- 4. 再按"全部打开" → 同样动作 PC 收到对应 0x16 等帧
- 5. 在 PC 道次输入框填 "4" → 按"道次关闭" → 只屏蔽第 4 道，其它 9 道照常工作

问题汇总

序号	项目	必/选	影响
1	0x1A D10 抢跳符号位	必修	抢跳无法识别
2	0x4B 电压上报	必修	电量监控不可用
3	0x42 子码 0x09	选修	参数双向同步
4	0x47 道次开/关	必修	"泳道控制"4 按钮硬件无反应
5	0x49/0x4A 冲突	PC 已修	已规避

协议手册全文详见同目录下 通讯协议变更说明_v2026.05.14.pdf。

生成时间：2026-05-14 · PC 端已 build verified